CSE422_Lab02_GA_02_v1

Sunday, February 15, 2026 10:10 AM

Genetic algorithms (GAs) are a class of optimization algorithms inspired by the principles of natural selection and genetics. These algorithms are used to solve search and optimization problems and are particularly effective for complex issues that are difficult to solve using traditional methods.

Genetic algorithms mimic the process of natural evolution, embodying the survival of the fittest among possible solutions. The core idea is derived from the biological mechanisms of reproduction, mutation, recombination, and selection. These biological concepts are translated into computational steps that help in finding optimal or near-optimal solutions to problems across a wide range of disciplines including engineering, economics, and artificial intelligence.

Genetic algorithms are used to evaluate large search spaces for a good solution. It is important to note that a genetic algorithm is not guaranteed to find the absolute best solution; it attempts to find the global best while avoiding local best solutions.

A *global best* is the best possible solution, and a *local best* is a solution that is less optimal. Figure 4.7 represents the possible best solutions if the solution must be minimized—that is, the smaller the value, the better. If the goal was to maximize a solution, the larger the value, the better. Optimization algorithms like genetic algorithms aim to incrementally find local best solutions in search of the global best solution.

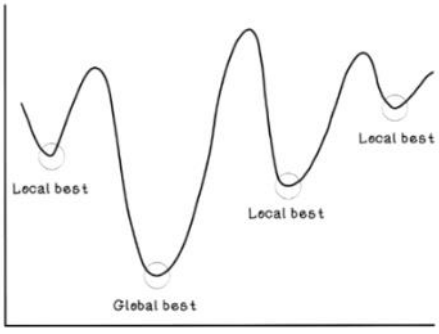


Figure 4.7 Local best vs. global best

The configuration for a genetic algorithm is based on the problem space. Each problem has a unique context and a different domain in which data is represented, and solutions are evaluated differently.

The general life cycle of a genetic algorithm is as follows:

- *Creating a population*—Creating a random population of potential solutions.
- *Measuring the fitness of individuals in the population*—Determining how good a specific solution is. This task is accomplished by using a fitness function that scores solutions to determine how good they are.
- *Selecting parents based on their fitness*—Selecting pairs of parents that will reproduce offspring.
- *Reproducing individuals from parents*—Creating offspring from their parents by mixing genetic information and applying slight mutations to the offspring.
- *Populating the next generation*—Selecting individuals and offspring from the population that will survive to the next generation.

1. Problem Overview

In this problem, we aim to simulate a simplified version of VLSI floorplanning within a chip circuit, a design challenge often found in the electrical chip manufacturing industry. Your objective is to determine the most optimal placement of 6 common chip components on a 2D chip grid, **minimizing** both **wire length** (that interconnects certain components)

In this problem, we aim to simulate a simplified version of VLSI floorplanning within a chip circuit, a design challenge often found in the electrical chip manufacturing industry. Your objective is to determine the most optimal placement of 6 common chip components on a 2D chip grid, **minimizing** both **wire length** (that interconnects certain components) and the overall required **chip area**, while **avoiding overlapping** chip components. Your task is to design a suitable chip layout optimizer using what you have been taught in your Genetic Algorithm class.

2. Grid and Components

Suppose the chip grid is of dimension **25 x 25** unit square (you are encouraged to take it as an input parameter to experiment later on how your algorithm behaves upon changing the grid size).

Say, the chip contains the following 6 functional blocks, each with a fixed size listed below:

Table 1

Block Name	Width (unit)	Height (unit)
ALU	5	5
Cache	7	4
Control Unit	4	4
Register File	6	6
Decoder	5	3
Floating Unit	5	5

3. Required Interconnections

One of the three goals of your layout design is to **minimize** the wiring distance between the following 6 connected component pairs:

Register File → ALU
Control Unit → ALU
ALU → Cache
Register File → Floating Unit
Cache → Decoder
Decoder → Floating Unit

You can assume all wiring connections are bidirectional, incur no additional space, and wires run **center-to-center** between components. Generally, the more compact the chip component placements are, the shorter the total wiring length becomes, and vice versa.

4. Fitness Objectives

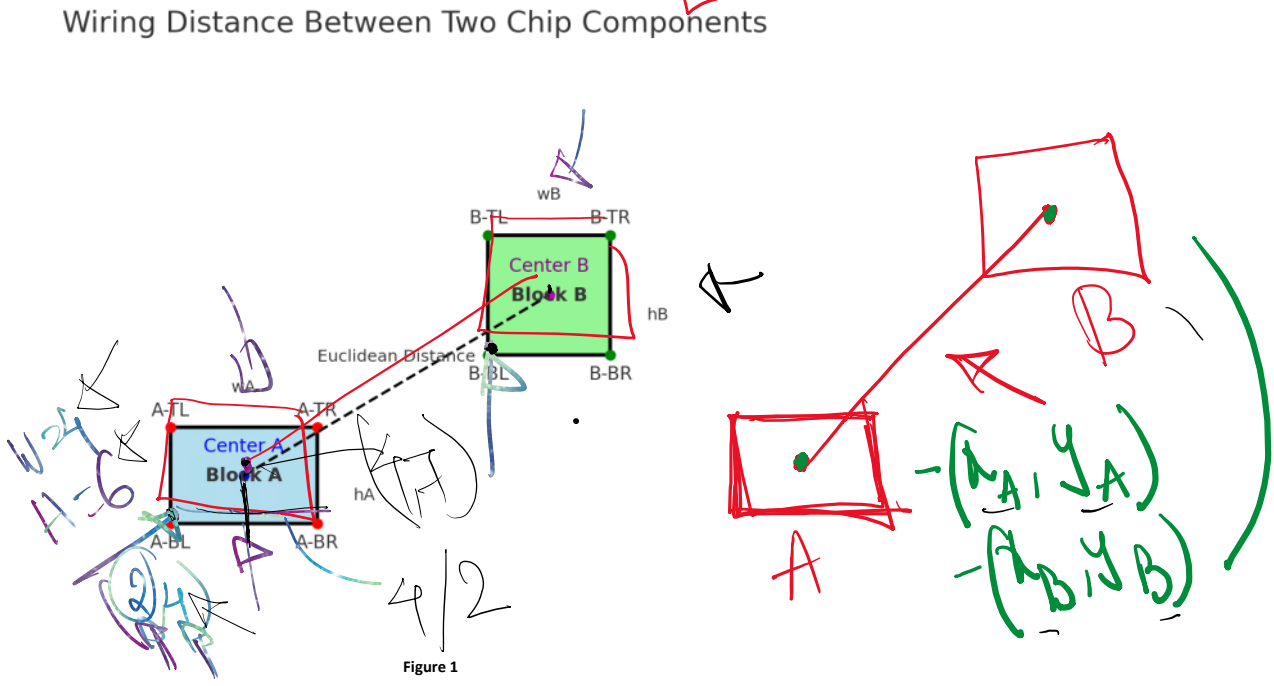


Figure 1

1. **Minimizing the total wiring distance:** Figure 1 above explains how the center-to-center wiring length can be calculated when both the blocks' height, width (wA, hA, wB, hB), and any single pair of corner points are given [suppose, bottom left of A (A-BL) and bottom left of B (B-BL) are given]. This is essentially the Euclidean Distance between two geometric points (two centers) in a 2D space. The **less total** wiring distance needed, the **more desirable** the layout is.

Xc = X + W/2
Yc = Y + H/2

d = Root((x1-x2)^2 + (y1-y2)^2)

Xc = X + W/2
Yc = Y + H/2

d = Root((x1-x2)^2 + (y1-y2)^2)

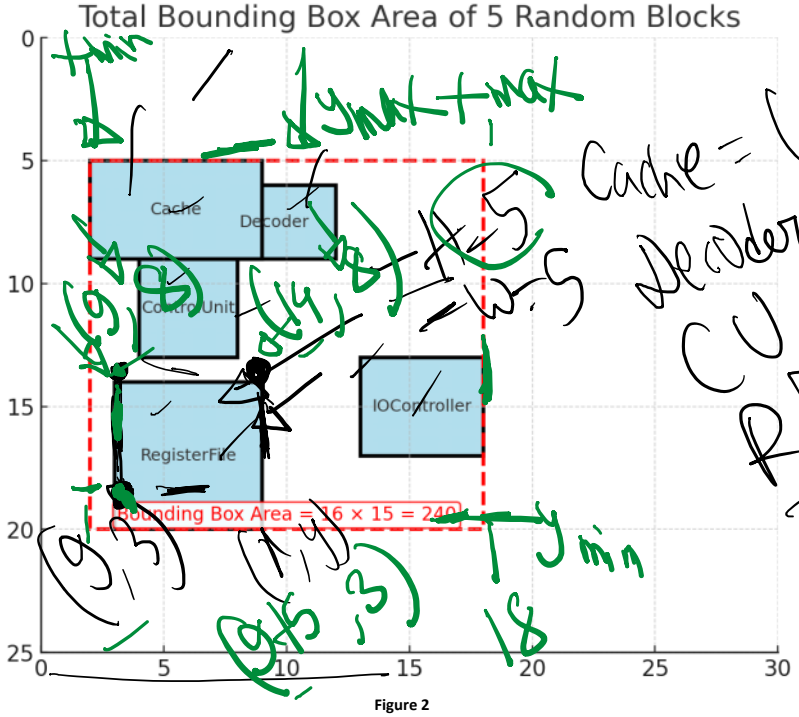


Figure 2

2. **Minimizing the total bounding area:** Figure 2 above hints at how the total bounding area can be calculated for a random placement of 5 component blocks. The red dashed rectangle is the bounding box- aka the smallest rectangle that contains all of the blocks inside. Given the bottom left coordinates (x,y) for all block components, you can easily find out the minimum and maximum of all x and y values, which will help you to calculate the area of such a bounding box [(x_max - x_min) * (y_max - y_min)]. The **smaller** the bounding box area is, the **better** the layout is.

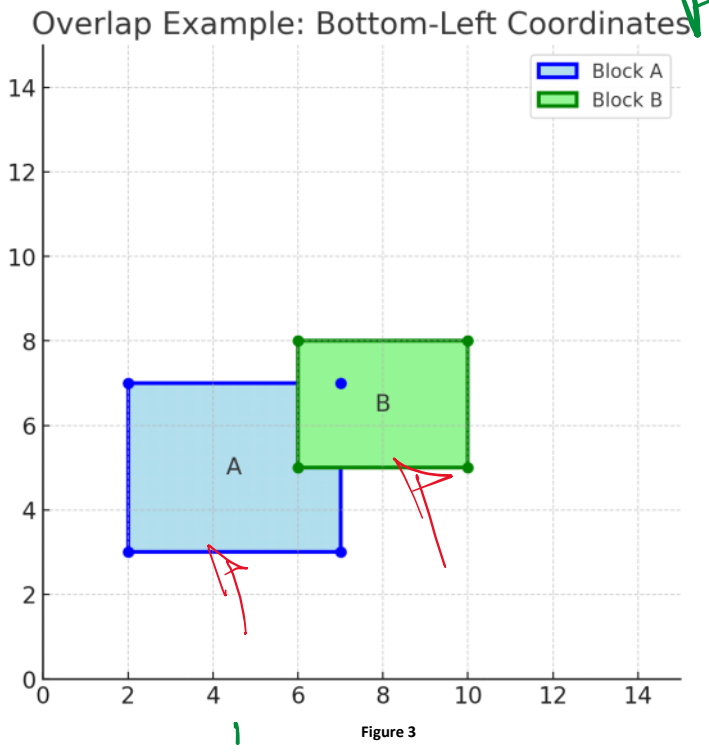


Figure 3

3. **Minimizing total component overlaps:** In Figure 2, there exists an overlap between the two components (cache and decoder). A single block can, in fact, be overlapped with multiple other blocks in a random placement. **Overlaps are not desirable.** Hence, a heavy penalty will be applied for each overlapping block pair. For any two block pairs A and B (Figure 3), one possible way to figure out the overlap is by checking the following condition:

```
overlap = not (
A_right <= B_left or
A  left >= B  right or
```

block pairs A and B (Figure 3), one possible way to figure out the overlap is by checking the following condition:

```
overlap = not (
A_right <= B_left or
A_left >= B_right or
A_bottom >= B_top or
A_top <= B_bottom)
```

where right, left, and top, bottom can be derived from the bottom-left x,y coordinate values of both blocks and the width and height of the corresponding blocks, respectively. The idea is that no overlap exists if one rectangle is completely to the left, right, above, **or** below the other

If ANY of these is true → rectangles are separated → no overlap.

Overlap exists only if NONE of the separation cases are true.

Two rectangles overlap if:

- A is not completely left of B
- AND A is not completely right of B
- AND A is not completely above B
- AND A is not completely below B

Task 1 Instructions

Start with a random initial population of 6 members, where each chromosome represents a candidate layout. Suppose the layout is specified by all the **bottom-left coordinates** of the 6 components (outlined in section 2 in that exact order). Check section 7 for a sample input analysis.

1. **Chromosome Representation / Encoding:**
   a. Devise a suitable encoding scheme to represent all the block components specified in the question. Note that the placement coordinate values will be in the range of [0, 25]. Be as concise as possible in your encoding; that is, try not to keep redundant information in your encoded version.
   b. Then generate an initial population of 6 chromosomes to start the GA loop, where each chromosome represents a placement strategy.

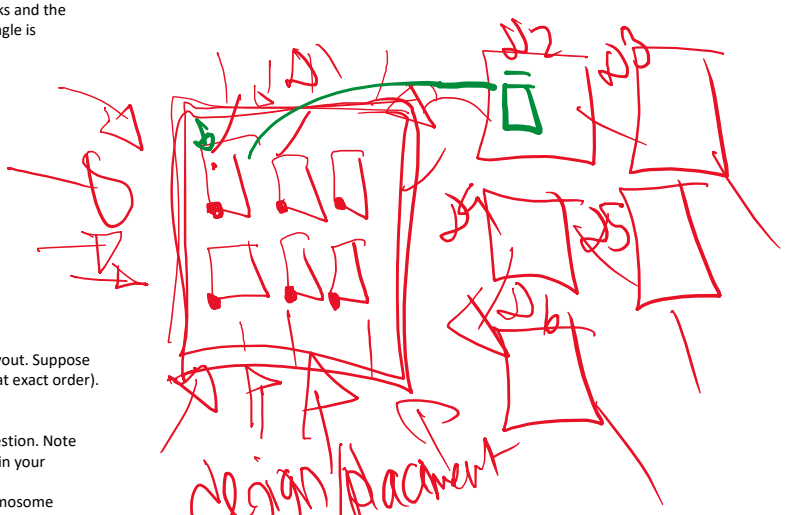
   A sample initial population set of 6 chromosomes, where each line below represents the bottom-left position coordinate of 6 chip component blocks:

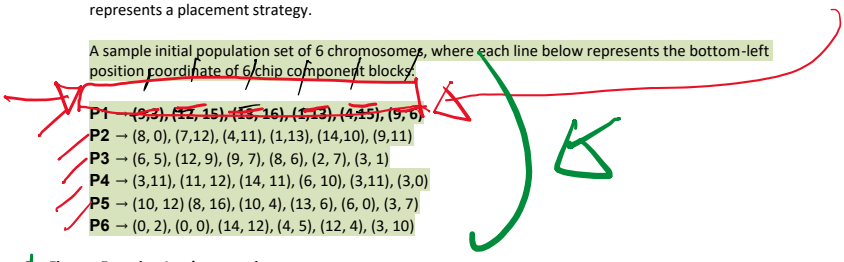
   P1 → (9,3), (12, 15), (13, 16), (1,13), (4,15), (9, 6)
   P2 → (8, 0), (7,12), (4,11), (1,13), (14,10), (9,11)
   P3 → (6, 5), (12, 9), (9, 7), (8, 6), (2, 7), (3, 1)
   P4 → (3,11), (11, 12), (14, 11), (6, 10), (3,11), (3,0)
   P5 → (10, 12) (8, 16), (10, 4), (13, 6), (6, 0), (3, 7)
   P6 → (0, 2), (0, 0), (14, 12), (4, 5), (12, 4), (3, 10)

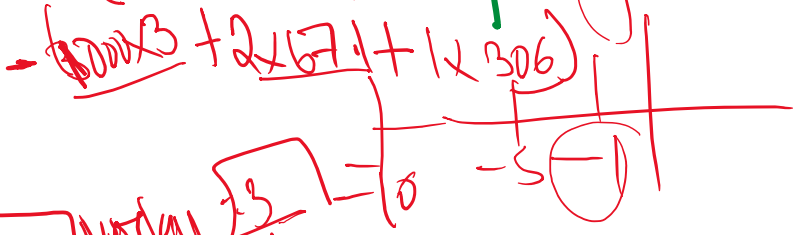
2. **Fitness Function Implementation:**
   a. Based on the 3 fitness objectives outlined above, design a suitable fitness function that focuses on penalizing bad placement combinations. Implementation hints are outlined in detail in section 4. Follow a prioritized order of these objectives as mentioned below, and reflect that in your fitness calculation.
      i. Overlap counts should be considered the least desirable, hence penalized way more than the other two
      ii. The total wiring distance and the total bounding area should be considered next
   b. Calculate the fitness of the chromosome pool using the hints specified above. A weighted sum can be a good fitness function to start with. Check the sample input output section for some ideas to start with.

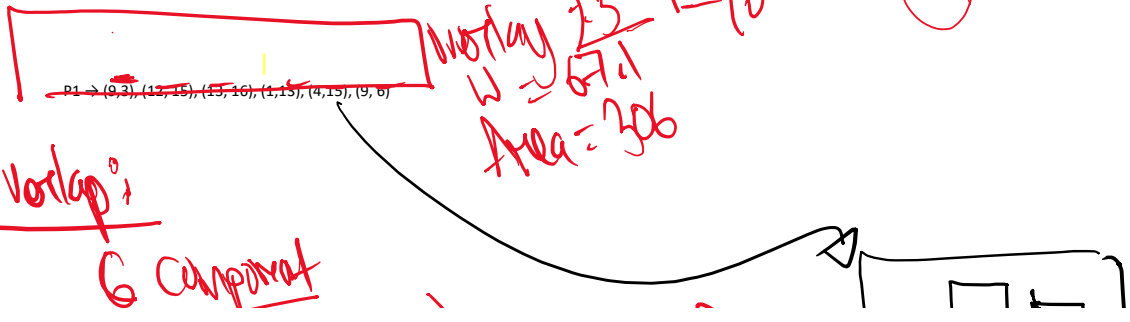
   Total fitness value =
   - (α * overlap penalty + β * wiring length penalty + γ * bounding area penalty)
   Considering, α = 1000, β = 2, γ = 1

Chromosome	Overlap	Wiring	Area	Fitness
P1	3	67.1	306	-3440.23
P2	3	57.7	342	-3457.40
P3	7	37.8	204	-7279.63
P4	5	53.3	240	-5346.51
P5	2	54.5	320	-2429.00
P6	3	52.6	288	-3393.18









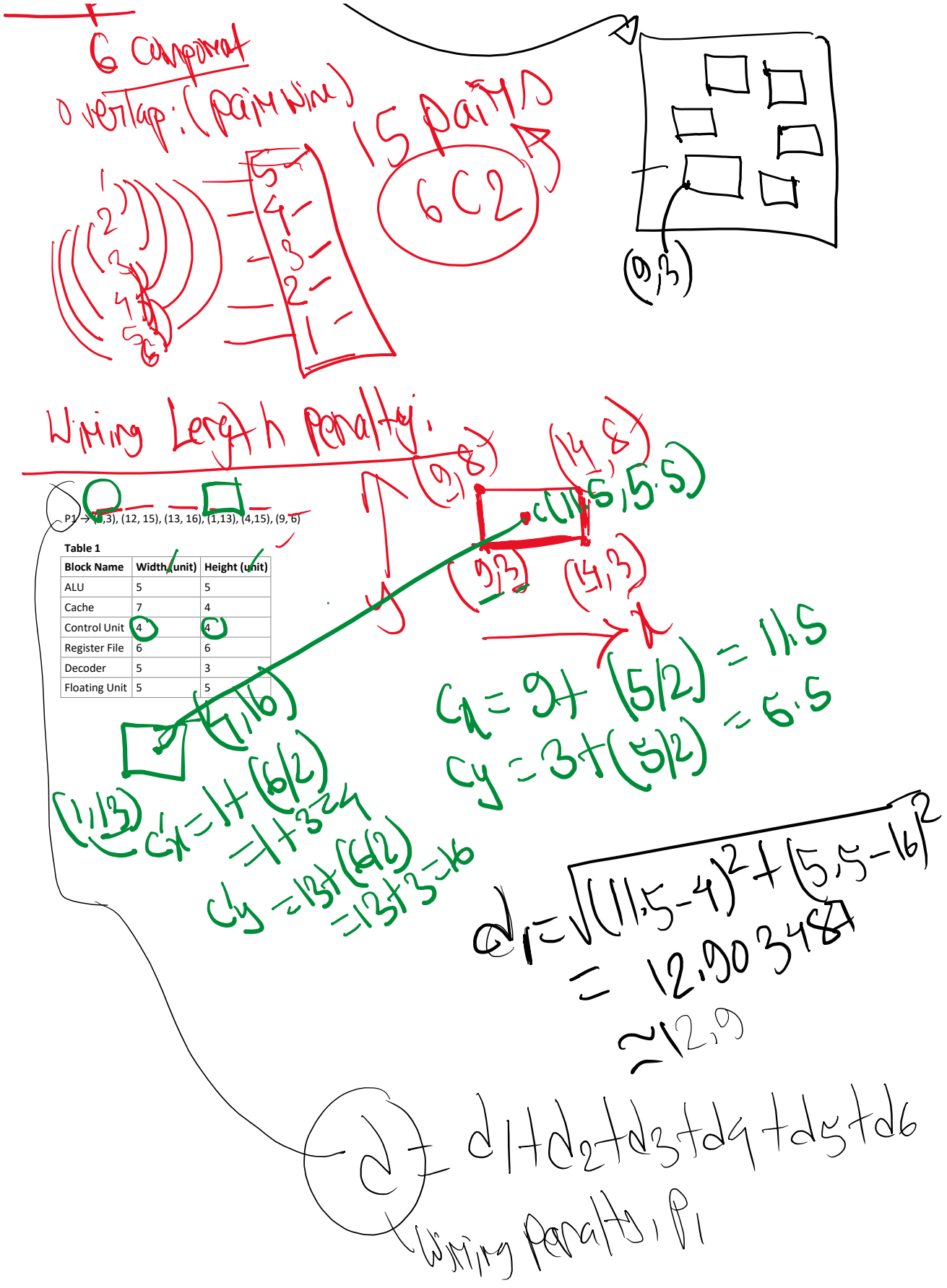


Table 1

Block Name	Width (unit)	Height (unit)
ALU	5	5
Cache	7	4
Control Unit	4	4
Register File	6	6
Decoder	5	3
Floating Unit	5	5

3. **Parent Selection:**
   a. Choose two parents based on random selection at each generation. Tournament selection, Roulette Wheel selection are some of the other popular selection techniques.
4. **Cross-over:**

3. **Parent Selection:**
   a. Choose two parents based on random selection at each generation. Tournament selection, Roulette Wheel selection are some of the other popular selection techniques.
4. **Cross-over:**
   a. Perform single-point crossover to create 2 new offspring from each pair of selected parents.
   b. To achieve this, pick a random split point from the chromosome representation of the parents and swap the complementary pieces to generate offspring.

Chromosome has 12 genes:
[x1,y1,x2,y2,...,x6,y6]
Pick a random split point.
Example: split at **k = 6** (after the 3rd block's y).

Parent gene lists

- P1 genes = (9,3)(12,15)(13,16) | (1,13)(4,15)(9,6)
- P2 genes = (8,0)(7,12)(4,11) | (1,13)(14,10)(9,11)

Offspring after crossover

Child C1 = first half of P1 + second half of P2
→ (9,3),(12,15),(13,16), (1,13),(14,10),(9,11)
Child C2 = first half of P2 + second half of P1
→ (8,0),(7,12),(4,11), (1,13),(4,15),(9,6)
Now evaluate them (same fitness formula):

- C1 fitness ≈ **-2419.88** (Overlap=2, Wiring≈56.94, Area=306)
- C2 fitness ≈ **-4372.33** (Overlap=4, Wiring≈62.66, Area=247)

Notice:

- C1 became **very strong** because overlap dropped to 2.
- C2 became worse because overlap increased to 4.

5. **Mutation:**
   a. Mutation ensures genetic diversity and prevents the algorithm from getting stuck in local optima.
   b. Implement the following mutation strategy to introduce random changes: pick any arbitrary component block from the chromosome representation and introduce random coordinate (i.e., the value of x and y) changes to that single particular block with a low probability (5-10% mutation rate).

Example mutation:

- Mutate Decoder (block 5) of C2 from (4,15) → (0,0)

So mutated C2 becomes:
→ (8,0),(7,12),(4,11), (1,13),(0,0),(9,6)
Re-evaluate:

- Mutated C2 fitness ≈ **-2415.51** (Overlap=2, Wiring≈74.76, Area=266)

6. **New Generation Creation:**
   a. Remember, the population size should be the same for all generations. Apply elitism to select new individuals (allow 1 or 2 elites to be carried forward to the next generation). Select the remaining candidates from the created offspring for the next generation.

Problem says allow 1 or 2 elites
Let's keep 1 elite:
Elite = P5 (fitness -2429.00)
So next generation already has:
NewPop = {P5}
We still need 5 more individuals.

7. **GA Loop:**
   a. Run genetic algorithms on such a population until the best fitness (a plateau) is achieved or the maximum number of 15 iterations is reached. Keep this generation count as a variable input for further experiments.
8. **Required Output:**
   a. Output the best possible placement strategy found once the algorithm halts. Deliverables include the best total fitness value, the corresponding total wiring length, the total bounding box area, and the total overlap counts. Also, decode the best chromosome representation to reflect the optimal placement of bottom-left coordinates.

Algorithm :

```
ALGORITHM GeneticAlgorithm_Floorplanning_Task1

Input:
  popSize = 6
  mutationRate ∈ [0.05, 0.10]
  α = 1000, β = 2, γ = 1
  maxGenerations = 15

Output:
  Best placement solution found

------------------------------------------------------------

Initialize population
  Population ← InitializeRandomPopulation(popSize)

Evaluate fitness of each chromosome
  For each chromosome G in Population:
    G.fitness ← Fitness(G)

generation ← 0
bestSolution ← BestIndividual(Population)

WHILE generation < maxGenerations DO

  // --- Elitism ---
  NewPopulation ← TopElite(Population, 1)

  WHILE size(NewPopulation) < popSize DO
```

```
    // --- Parent Selection (Random) ---
    parent1, parent2 ← RandomSelect(Population)

    // --- Single-Point Crossover ---
    child1, child2 ← SinglePointCrossover(parent1, parent2)

    // --- Mutation (5–10%) ---
    child1 ← Mutate(child1, mutationRate)
    child2 ← Mutate(child2, mutationRate)

    // --- Evaluate offspring ---
    child1.fitness ← Fitness(child1)
    child2.fitness ← Fitness(child2)

    Add child1 to NewPopulation if space available
    Add child2 to NewPopulation if space available

  END WHILE

  Population ← NewPopulation

  currentBest ← BestIndividual(Population)

  IF currentBest.fitness > bestSolution.fitness THEN
    bestSolution ← currentBest
  END IF

  generation ← generation + 1

END WHILE

RETURN bestSolution

END ALGORITHM
```

Task 2

Take a look at **Step 4 of Task 1**. Can you incorporate the **Two-point crossover mechanism** into it?

- To implement this task, randomly select two parents from your initial population. Then perform a two-point crossover to generate two children. The two points have to be chosen randomly, but it has to be ensured that the second point always comes after the first point.

Sample Input and Output

A sample initial population set of 6 chromosomes, where each line below represents the bottom-left position coordinate of 6 chip components blocks:

P1 → **(9,3), (12, 15), (13, 16), (1,13), (4,15), (9, 6)**
P2 → (8, 0), (7,12), (4,11), (1,13), (14,10), (9,11)
P3 → (6, 5), (12, 9), (9, 7), (8, 6), (2, 7), (3, 1)
P4 → (3,11), (11, 12), (14, 11), (6, 10), (3,11), (3,0)
P5 → (10, 12) (8, 16), (10, 4), (13, 6), (6, 0), (3, 7)
P6 → (0, 2), (0, 0), (14, 12), (4, 5), (12, 4), (3, 10)

For **P1,**

Pairwise block overlap count = 3

Total wiring distance (center-to-center) of the specified connected pairs = 12.91 + 12.98 + 12.18 + 10.61 + 9.01 + 9.43 = 67.1

Total bounding box area = (x_max - x_min) * (y_max - y_min) = (19-1) * (20-3) = 306

Total fitness value = - (alpha * overlap penalty + beta * wiring length penalty + gamma * bounding area penalty)
Considering, alpha = 1000, beta = 2, gamma = 1:

Fitness for generation 1 chromosome 1 = - (1000 * 3) - (67.1 * 2) - 306 = -3440.23